

Day 58: Backpropagation Through Time

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I
(Day 3 of 7)

0. Where Today Fits

Day 57 built the vanilla RNN's forward pass and verified it without any gradients. Today builds the backward pass -- the first time this network can actually LEARN -- verifies it with a numeric gradient check, then trains it twice. The second training run overturns a prediction Day 57 set up.

How to read each section

Every experiment below is laid out in the same order: **Specification** (exactly what was set up -- data, sizes, settings, what counts as success), **Why this test** (what question it answers), **Implementation** (the real code from day58_rnn_bptt.py that produces the result), **Actual output** (copied verbatim from day58_run_output.txt), and **What it means**. You should be able to predict the output from the first three before you read it.

Watch: [L15.4 Backpropagation Through Time Overview](#)

Watch: [Lecture 14: Intro to RNNs & BackProp-Through-Time \(BPTT\)](#)

1. The Model and the BPTT Equations

SPECIFICATION

Model	VanillaRNNClassifier -- many-to-one: reads the whole sequence, classifies from the FINAL hidden state only
Input	X of shape (batch m, length T): one scalar per timestep
Recurrence	$h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$, with $h_{-1} = h_0 = \text{zeros}$
Output layer	$\text{probs} = \text{softmax}(h_{T-1} @ W_{hy} + b_y)$
Parameters	W_{xh} (1 x H), W_{hh} (H x H), b_h (H), W_{hy} (H x 2), b_y (2), where H = hidden_dim
Loss	mean cross-entropy over the batch
Update rule	plain SGD: $p \leftarrow p - \text{lr} * \text{dp}$, applied inside backward()

Why this matters

Ordinary backprop computes each weight's gradient once, because each weight is used once. Here W_{xh} , W_{hh} , and b_h are used at EVERY timestep, so the loss reaches them through T separate paths. The chain rule says the total gradient is the SUM over those paths -- that summing, plus handing gradient from one timestep back to the previous one, is all BPTT adds to ordinary backprop.

MATH

Output layer (once): $d\text{logits} = \text{probs} - y_{\text{onehot}}$; $d\text{Why} = h_{T-1}^T @ d\text{logits} / m$; $dh = d\text{logits} @ \text{Why}^T$

Then for $t = T-1$ down to 0:

```
da_t = dh * (1 - h_t^2)    undo tanh (its derivative is 1 - tanh^2, and tanh(a_t) is h_t)
dWxh += x_t.T @ da_t / m   ACCUMULATE
dWhh += h_{t-1}.T @ da_t / m ACCUMULATE -- Whh multiplied the PREVIOUS state
dbh += sum(da_t) / m       ACCUMULATE
dh = da_t @ Whh.T          hand gradient back to h_{t-1} -- the 'through time' step
```

IMPLEMENTATION

```
def rnn_backward(dh_last, X, H, h0, Whh):
    m, T = X.shape
    dWxh = np.zeros((1, Whh.shape[0]))    # accumulators start at zero --
    dWhh = np.zeros_like(Whh)              # each receives one contribution
    dbh = np.zeros(Whh.shape[0])           # PER timestep
    dh = dh_last                            # gradient from the output layer
    for t in reversed(range(T)):             # T-1, T-2, ..., 0
        h_t = H[:, t, :]
        h_prev = H[:, t - 1, :] if t > 0 else h0
        x_t = X[:, t:t + 1]
        da_t = dh * (1.0 - h_t ** 2)        # undo tanh
        dWxh += x_t.T @ da_t / m            # ACCUMULATE
        dWhh += h_prev.T @ da_t / m         # ACCUMULATE (h_prev, not h_t)
        dbh += da_t.sum(axis=0) / m         # ACCUMULATE
        dh = da_t @ Whh.T                   # back to h_{t-1}
    return dWxh, dWhh, dbh

# inside VanillaRNNClassifier.backward():
dlogits = self.probs - y_onehot
dwhy = self.h_last.T @ dlogits / m
dby = dlogits.sum(axis=0) / m
dh_last = dlogits @ self.Why.T             # BPTT's starting point
dWxh, dWhh, dbh = rnn_backward(dh_last, self.X, self.H, self.h0, self.Whh)
```

The forward pass caches X , every hidden state H , and h_0 -- exactly what this loop reads back. The last line of the loop overwrites dh (not $+=$) because in a many-to-one network no other gradient enters h_{t-1} ; a network with an output at every timestep would add that timestep's output gradient here.

WHY THIS CHOICE

Two mistakes BPTT punishes silently: using h_t where h_{t-1} belongs in the $dWhh$ line, and writing $=$ instead of $+=$ (keeping only one timestep's contribution). Both still produce gradients of the right SHAPE -- only a numeric check catches them. That is Section 2.

2. Gradient Check -- Is BPTT's Math Right?

SPECIFICATION

Network	VanillaRNNClassifier(hidden_dim=4, n_classes=2, lr=0.01, seed=1)
Input	5 random sequences of length 6: $X = \text{randn}(5, 6) * 0.8$ (seed 1)
Labels	[0, 1, 0, 1, 1], one-hot encoded
Checked	all 5 parameters -- Wxh, Whh, bh, Why, by -- 3 random entries each = 15 checks
Numeric gradient	central difference: $(L(p + \text{eps}) - L(p - \text{eps})) / (2 * \text{eps})$, $\text{eps} = 1e-5$
Compared by	relative error = $ \text{numeric} - \text{analytic} / (\text{numeric} + \text{analytic})$
Pass criterion	max relative error < $1e-5$

Why this test

The numeric gradient needs no calculus: it literally nudges one weight up and down and watches the loss change. If the hand-derived BPTT agrees with it on every parameter, the derivation and the code are both right. A 6-step sequence is used so that every timestep's contribution to the accumulated gradients actually matters -- a bug that dropped one timestep would show up.

IMPLEMENTATION

```
net.forward(X)
net.backward(y_oh)                # computes grads AND applies an SGD step
for name in ["Wxh", "Whh", "bh", "Why", "by"]:
    getattr(net, name)[...] += net.lr * net.grads[name]    # undo the step

for name in ["Wxh", "Whh", "bh", "Why", "by"]:
    param = getattr(net, name)
    for _ in range(3):
        idx = tuple(rng.randint(0, s) for s in param.shape)
        orig = param[idx]
        param[idx] = orig + eps; lp = cross_entropy_loss(net.forward(X), y_oh)
        param[idx] = orig - eps; lm = cross_entropy_loss(net.forward(X), y_oh)
        param[idx] = orig
        numeric = (lp - lm) / (2 * eps)
        ana = net.grads[name][idx]
        rel_err = abs(numeric - ana) / max(abs(numeric) + abs(ana), 1e-8)
```

backward() both computes gradients and updates the weights, so the update is reversed first -- the numeric check must perturb the SAME weights the analytic gradient was computed at.

ACTUAL OUTPUT

param	index	analytic	numeric	rel_err
wxh	(0, 3)	0.00740539	0.00740539	8.70e-09
wxh	(0, 1)	0.00600723	0.00600723	1.44e-08
wxh	(0, 0)	0.00911978	0.00911978	2.17e-08
whh	(3, 1)	0.01986643	0.01986643	4.73e-09
whh	(1, 3)	-0.00869595	-0.00869595	1.06e-09
whh	(2, 0)	0.04972201	0.04972201	1.02e-08
bh	(0,)	-0.13430024	-0.13430024	9.63e-09
bh	(2,)	0.20738258	0.20738257	9.16e-09
bh	(2,)	0.20738258	0.20738257	9.16e-09
why	(3, 1)	-0.15177114	-0.15177114	1.55e-08
why	(3, 1)	-0.15177114	-0.15177114	1.55e-08
why	(1, 1)	-0.08811112	-0.08811112	1.63e-08
by	(1,)	-0.21781345	-0.21781345	1.10e-08
by	(0,)	0.21781345	0.21781345	1.10e-08
by	(0,)	0.21781345	0.21781345	1.10e-08

max relative error across all checked entries: 2.17e-08 (PASS -- BPTT is correct)

What it means

Every entry agrees to about eight significant figures, including all of Wxh, Whh, and bh -- the three parameters whose gradients are accumulated across time. (Some indices repeat because they are drawn at random.) The math is right; the next question is whether the network can use it to learn.

3. Does It Actually Learn? -- Training at T=10

SPECIFICATION

Task	Day 56's order detection: one +3 spike and one -3 spike at two random positions; label 1 if the +3 comes first. Background noise: Gaussian, std 0.3
Sequence length	T = 10, spike positions drawn from 0..9
Data	train: 300 sequences (150 per class, seed 42); test: 200 (100 per class, seed 999)
Network	VanillaRNNClassifier(hidden_dim=16, lr=0.1, seed=7)
Training	300 epochs, mini-batch size 30, reshuffled every epoch, plain SGD
Measured	test accuracy before training, train and test accuracy after, loss at first and last epoch
Chance level	accuracy 0.5; loss $\ln(2) = 0.693$

Why this test

A passed gradient check proves the math, not the usefulness. This is the first time an RNN in this series is trained at all, on exactly the task where Day 56's dense network collapsed to chance and its CNN was capped by its kernel. T=10 is short -- a length where Day 57 still measured healthy sensitivity.

IMPLEMENTATION

```
def train(net, X_train, y_train, epochs, batch_size, seed):
    rng = np.random.RandomState(seed)
    for epoch in range(epochs):
        order = rng.permutation(len(y_train))          # fresh shuffle
        for start in range(0, len(order), batch_size):
            idx = order[start:start + batch_size]
            y_oh = one_hot(y_train[idx], 2)
            net.forward(X_train[idx])                  # forward
            net.backward(y_oh)                          # BPTT + SGD step

T = 10
X_train, y_train = make_order_dataset(150, T, 0, T - 1, seed=42)
X_test, y_test = make_order_dataset(100, T, 0, T - 1, seed=999)
net = VanillaRNNClassifier(hidden_dim=16, lr=0.1, seed=7)
acc_before = net.accuracy(X_test, y_test)
train(net, X_train, y_train, epochs=300, batch_size=30, seed=7)
acc_after = net.accuracy(X_test, y_test)
```

ACTUAL OUTPUT

```
test_acc BEFORE training (random weights): 0.3950
train_acc AFTER training:                  1.0000
test_acc AFTER training:                   1.0000
training loss: 0.7331 (epoch 0) -> 0.0004 (epoch 299)
```

What it means

From below chance (0.395 -- random weights happen to lean the wrong way) to a perfect 1.000 on unseen test sequences, with the loss driven from above chance (0.733) to nearly zero (0.0004). Weights shared across time plus state carried forward solve what Day 56's architectures could not.

4. The Surprise -- T=40 Trains Too

SPECIFICATION

Change from Section 3	ONLY the sequence length: $T = 40$, spike positions drawn from 0..39. Same data sizes and seeds, same network, lr, epochs, batch size
Prediction being tested	Day 57 measured the sensitivity of h_T to x_0 collapsing to 0.000024 at $T=40$ -- so training should struggle
Memory distance	for each test sequence: $(T - 1)$ minus the position of whichever spike came FIRST -- how many steps that spike must be remembered until classification
Small-signal sensitivity	ONE noise-only sequence (std 0.3, seed 0); nudge x_0 by $\pm 1e-3$; report $ h_T(+)-h_T(-) / (2 * 1e-3)$. This is Day 57's measure
Large-signal effect	50 noise-only sequences (seed 0); add a full +3 spike at x_0 ; report the mean $ h_T(\text{with spike}) - h_T(\text{without}) $. No division -- a raw difference
When measured	both sensitivities BEFORE and AFTER training, on the identical sequences
Memory diagnostics	fraction of final hidden units with $ h > 0.95$ (tanh saturation), and the largest eigenvalue of Whh

Why this test

Day 57 framed its collapsing sensitivity as a preview of vanishing gradients. The honest thing to do with a prediction is to test it: change nothing but the length. And if the result contradicts the prediction, don't just report it -- find out why. The memory-distance breakdown checks whether the network is cheating on easy short-range examples; the two sensitivity measures and the memory diagnostics test a specific explanation.

IMPLEMENTATION

```
def small_signal_sensitivity(net, T, seed):          # Day 57's measure: a local derivative
    X = np.random.RandomState(seed).randn(1, T) * 0.3
    h0 = np.zeros((1, net.hidden_dim)); eps = 1e-3
    Xp = X.copy(); Xp[0, 0] += eps
    Xm = X.copy(); Xm[0, 0] -= eps
    hp = rnn_forward(Xp, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    hm = rnn_forward(Xm, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    return np.linalg.norm(hp - hm) / (2 * eps)

def large_signal_effect(net, T, seed, n=50):        # a task-sized spike, not a nudge
    X = np.random.RandomState(seed).randn(n, T) * 0.3
    Xs = X.copy(); Xs[:, 0] += 3.0
    h0 = np.zeros((n, net.hidden_dim))
    h_plain = rnn_forward(X, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    h_spike = rnn_forward(Xs, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    return np.linalg.norm(h_spike - h_plain, axis=1).mean()

small_before = small_signal_sensitivity(net, T, seed=0) # BEFORE training
large_before = large_signal_effect(net, T, seed=0)
train(net, X_train, y_train, epochs=300, batch_size=30, seed=7)
small_after = small_signal_sensitivity(net, T, seed=0)  # AFTER, same sequences
large_after = large_signal_effect(net, T, seed=0)

first_spike = np.array([min(np.argmax(s), np.argmin(s)) for s in X_test])
mem_dist = (T - 1) - first_spike
saturated = (np.abs(net.h_last) > 0.95).mean()
spectral_radius = np.max(np.abs(np.linalg.eigvals(net.Whh)))
```

The two sensitivity functions differ in exactly one idea: the first divides by 2ϵ to estimate a derivative (valid only for infinitesimal changes); the second applies a change the size of the task's real signal and reports the raw effect. For a linear system they would tell the same story.

ACTUAL OUTPUT

```
train_acc AFTER training (T=40): 1.0000
test_acc AFTER training (T=40):  0.9950
training loss: 0.9365 (epoch 0) -> 0.0004 (epoch 299)
```

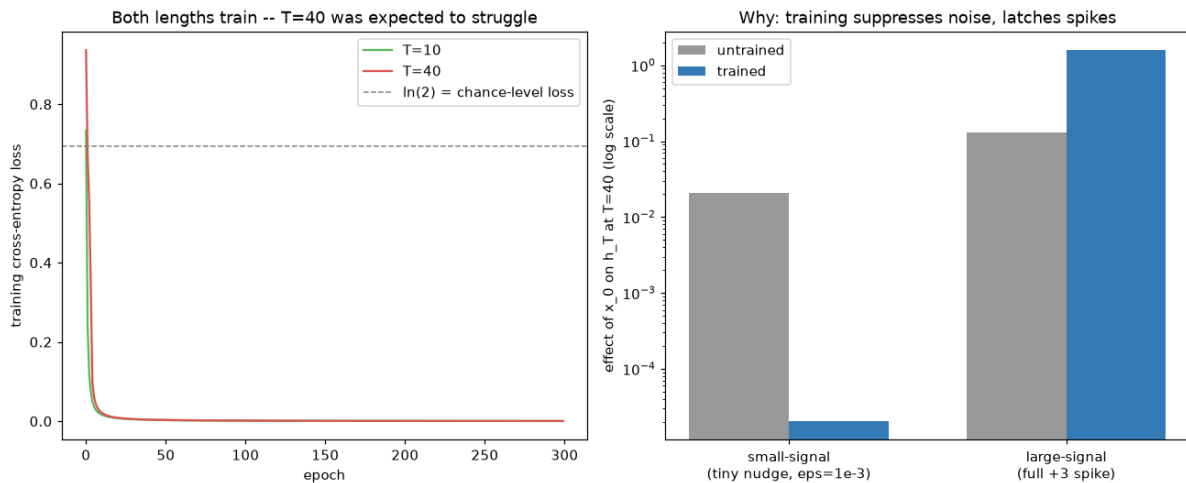
ACTUAL OUTPUT

```
test accuracy by memory distance (steps the first spike must be carried to the end):
 0-9 steps: n= 9   test_acc=1.0000
10-19 steps: n= 43  test_acc=1.0000
20-29 steps: n= 58  test_acc=1.0000
30-39 steps: n= 90  test_acc=0.9889
```

ACTUAL OUTPUT

measure (h_T at T=40, x_0 perturbed)	untrained	trained
small-signal: $ dh_T/dx_0 $, $\epsilon=1e-3$	0.020786	0.000021
large-signal: $ h_T(+3 \text{ spike}) - h_T(\text{none}) $	0.131794	1.582471

trained network: 19.8% of final hidden units have $|h| > 0.95$; largest $|eigenvalue|$ of Whh = 2.168



What it means

The prediction was wrong. $T=40$ reaches 0.995, and the memory-distance table rules out cheating: accuracy is still 0.989 when the first spike must be carried 30-39 steps. The sensitivity table explains why -- the two measures move in OPPOSITE directions during training. Sensitivity to a tiny nudge at x_0 drops about 1000x; the effect of a full +3 spike grows about 12x. The network learned to ignore small noise and LATCH onto large spikes: about a fifth of its final hidden units sit pinned near tanh's ± 1 limits, and Whh's largest eigenvalue grew to 2.17. In a linear recurrence an eigenvalue above 1 would explode, but tanh bounds every unit, so saturated units become stable memory instead.

WHY THIS CHOICE

Day 57's measurement wasn't false -- it was the wrong instrument for this task. A tiny-epsilon finite difference is a LOCAL, LINEAR measurement. Latching is a large-signal, NONLINEAR effect: a big input flips a unit into saturation and a strong self-connection holds it there. Shrinking epsilon can never reveal it. General lesson: measure at the scale of the signal the task actually carries. And the caveat for Day 59: this task is unusually latch-friendly (+3 spikes over 0.3 noise, one binary fact to store). Vanishing gradients are real; this task routes around them, so Day 59 needs one that doesn't.

5. What This Maps To in PyTorch

IMPLEMENTATION

```
class RNNClassifier(nn.Module):
    def __init__(self, hidden_dim=16, n_classes=2):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_dim,
                           nonlinearity='tanh', batch_first=True)
        self.out = nn.Linear(hidden_dim, n_classes)

    def forward(self, x):
        # x: (batch, T, 1)
        H, h_last = self.rnn(x)
        return self.out(h_last[-1])  # many-to-one

loss.backward()  # autograd runs BPTT -- the same accumulate-across-time loop
```

Interview Preparation -- Technical Questions

Q: Why must an RNN's weight gradients be accumulated across timesteps rather than computed once?

A: The same W_{xh} , W_{hh} , and b_h are used at every timestep, so the loss depends on each through T separate paths. By the multivariable chain rule the total gradient is the sum of each path's contribution -- which is why `rnn_backward` uses `+=` at every step.

Q: In the dW_{hh} update, why is it h_{t-1} and not h_t ?

A: In the forward pass W_{hh} multiplies the PREVIOUS state: $a_t = x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h$. A weight's gradient is (the input it multiplied) $^T @$ (the gradient at its output), so step t uses h_{t-1} -- and h_0 at $t = 0$.

Q: Why can the tanh derivative be computed from h_t alone?

A: $d \tanh(a)/da = 1 - \tanh(a)^2$, and $\tanh(a_t)$ is exactly the cached h_t , so $(1 - h_t^2)$ needs no extra storage.

Q: Day 57's small-signal sensitivity at $T=40$ was tiny, yet the RNN trained to 0.995. Explain.

A: The small-signal measure is a local derivative. The trained network latches onto LARGE inputs: a $+3$ spike saturates tanh units and recurrent weights with an eigenvalue above 1 hold them there. Measured directly, training cut small-signal sensitivity $\sim 1000\times$ while growing large-signal effect $\sim 12\times$.

Interview Preparation -- Theory Questions

Q: What does it mean to 'unroll' an RNN, and why does that make BPTT ordinary backprop?

A: Unrolling writes the recurrence out as T copies of the same cell in a chain -- a deep feedforward network whose layers share weights. Backprop applies unchanged, plus the rule that shared weights sum their gradients.

Q: How can a recurrent weight matrix with spectral radius above 1 avoid exploding?

A: In a linear recurrence it would grow without bound, but tanh squashes every unit into $(-1, 1)$. Units driven into saturation can become stable fixed points -- exactly how this lesson's trained network stores memory.

Q: If this task trains fine at $T=40$, is the vanishing-gradient problem overstated?

A: No. This task is unusually easy to remember -- large spikes over small noise, one binary fact. Tasks with subtle signals, or more to remember, can't rely on latching, and there gradients through long chains of tanh and W_{hh} really do shrink. That is Day 59's topic.

Key Takeaway and What's Next

Key takeaway: BPTT is ordinary backprop on the unrolled network, plus one rule: shared weights sum their gradients across every timestep. It passed a gradient check on all five parameters and trained the RNN to 100% at $T=10$. It also trained to 99.5% at $T=40$ -- overturning Day 57's prediction, because the network learned to latch onto large spikes, a nonlinear memory mechanism a small-signal derivative cannot detect.

What's next: Day 59 demonstrates vanishing and exploding gradients for real -- on a task chosen so that latching can't rescue the network.

Suggested watch order: 1) L15.4 Backpropagation Through Time Overview, 2) Lecture 14: Intro to RNNs & BPTT.